SCREENSHOT EVIDENCE · FINAL DELIVERABLE

Advanced Shell Simulator

Annotated screenshots and implementation notes

A guided explanation of the three screenshots documenting the integrated shell, including piping, user authentication, file permissions, scheduling, paging, synchronization, and simulated background jobs.

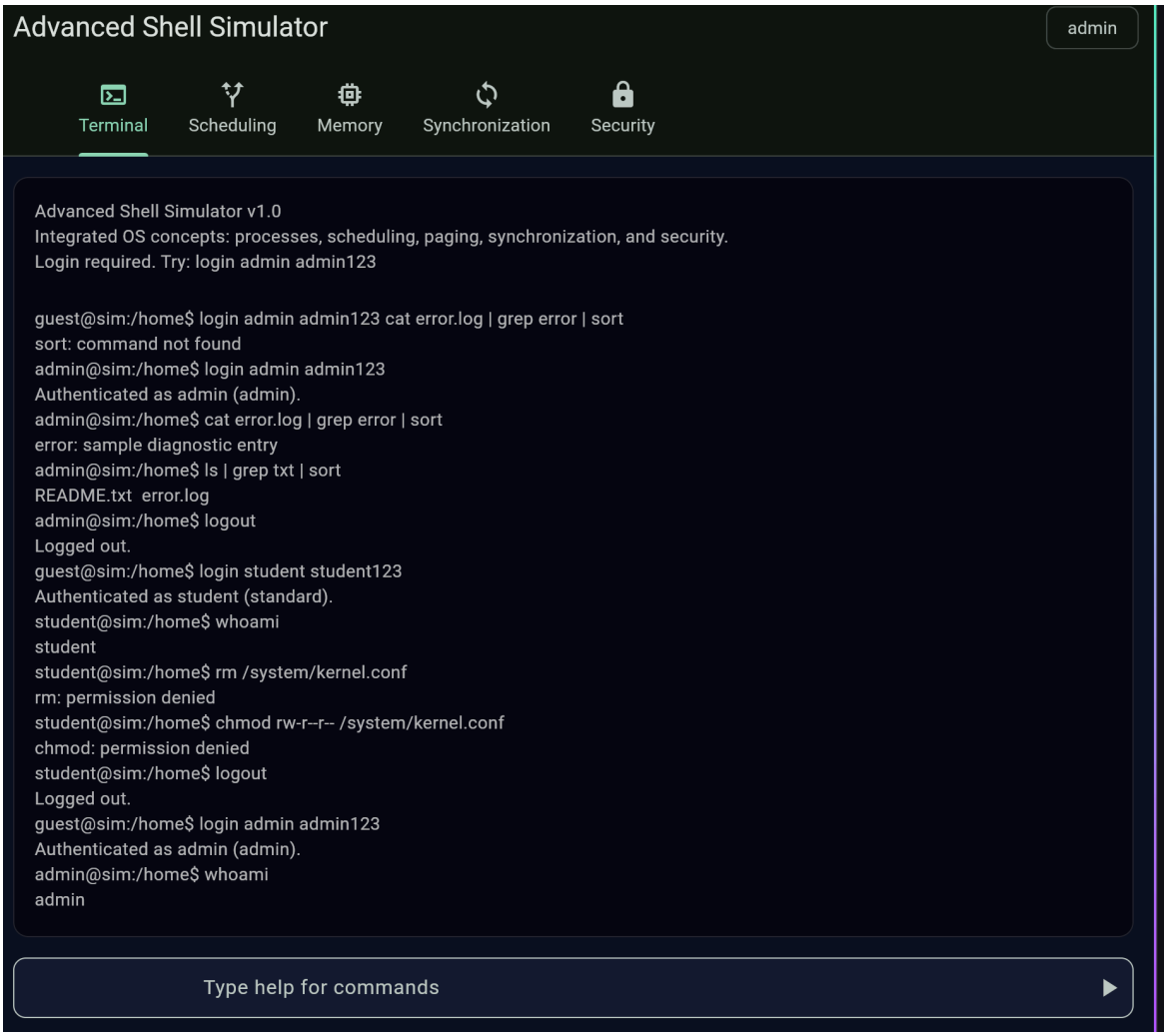
How to read this report: each evidence page reproduces a screenshot from the supplied document and explains the visible commands and results. Notes distinguish simulated behavior from real operating-system activity and identify relevant limitations.

Project: Advanced Shell Simulator · Flutter application with in-memory OS concept simulations

1. Terminal, piping, authentication, and permissions

Advanced Shell Simulator — Annotated Screenshot Report

This screenshot records both command-pipeline testing and role-based access checks in the integrated shell simulator.

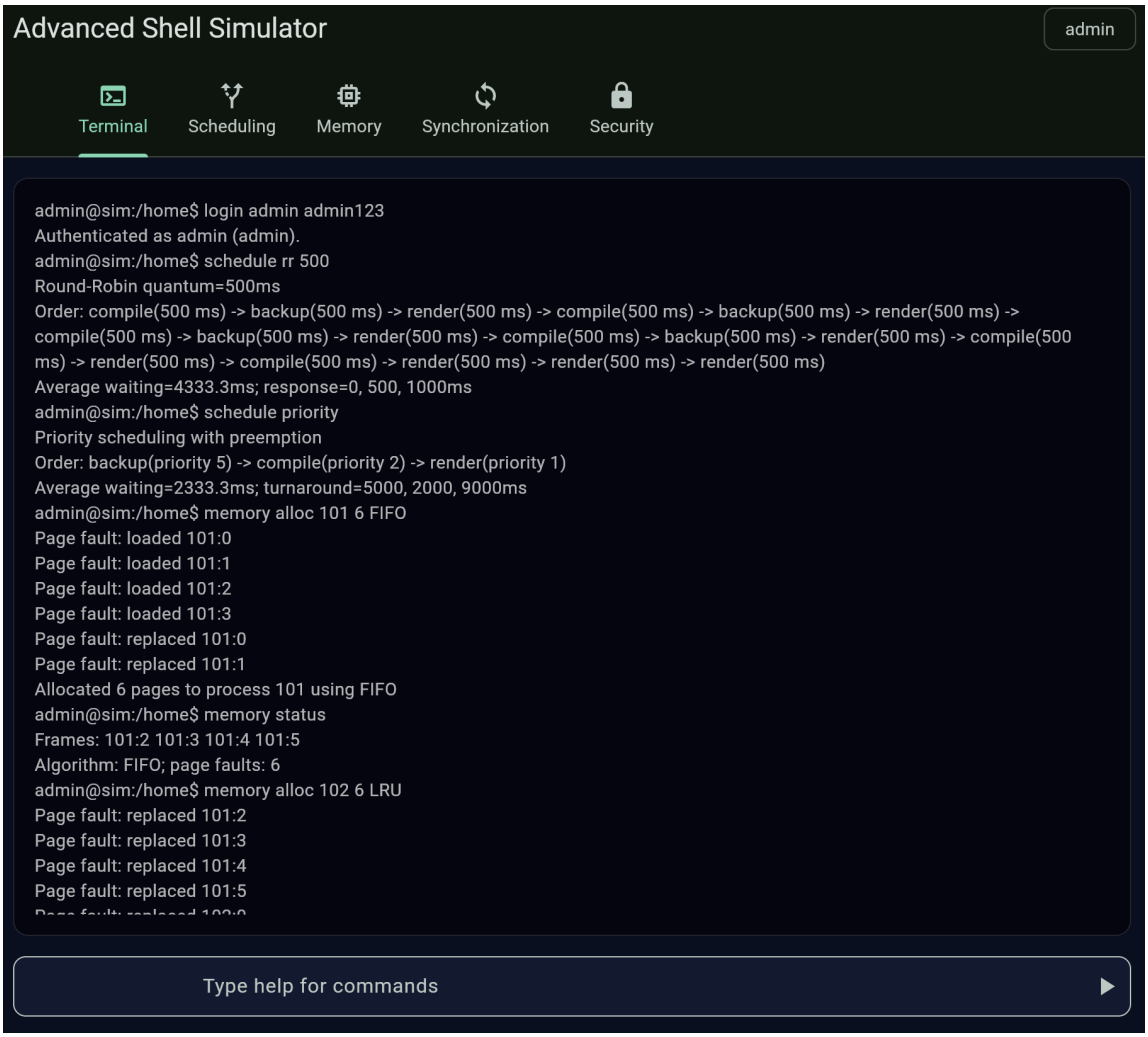


What this screenshot demonstrates

- **Authentication:** the terminal shows a successful admin login, logout, and a separate student login. The `whoami` command confirms which simulated user is active.
- **Pipeline:** after logging in first, `cat error.log | grep error | sort` displays the matching diagnostic line. Each stage receives the previous stage's output.
- **File protection:** as the standard user, both removal of `/system/kernel.conf` and changing its mode are rejected with "permission denied."
- **Observed limitation:** the first combined login-and-pipeline attempt produces an error; run login as a separate command before protected commands. Also, `ls` prints entries together on one line, so the shown `ls | grep txt` output does not filter entries individually.

2. CPU scheduling and page replacement

The terminal demonstrates the simulator's scheduling calculations and a four-frame paging model.

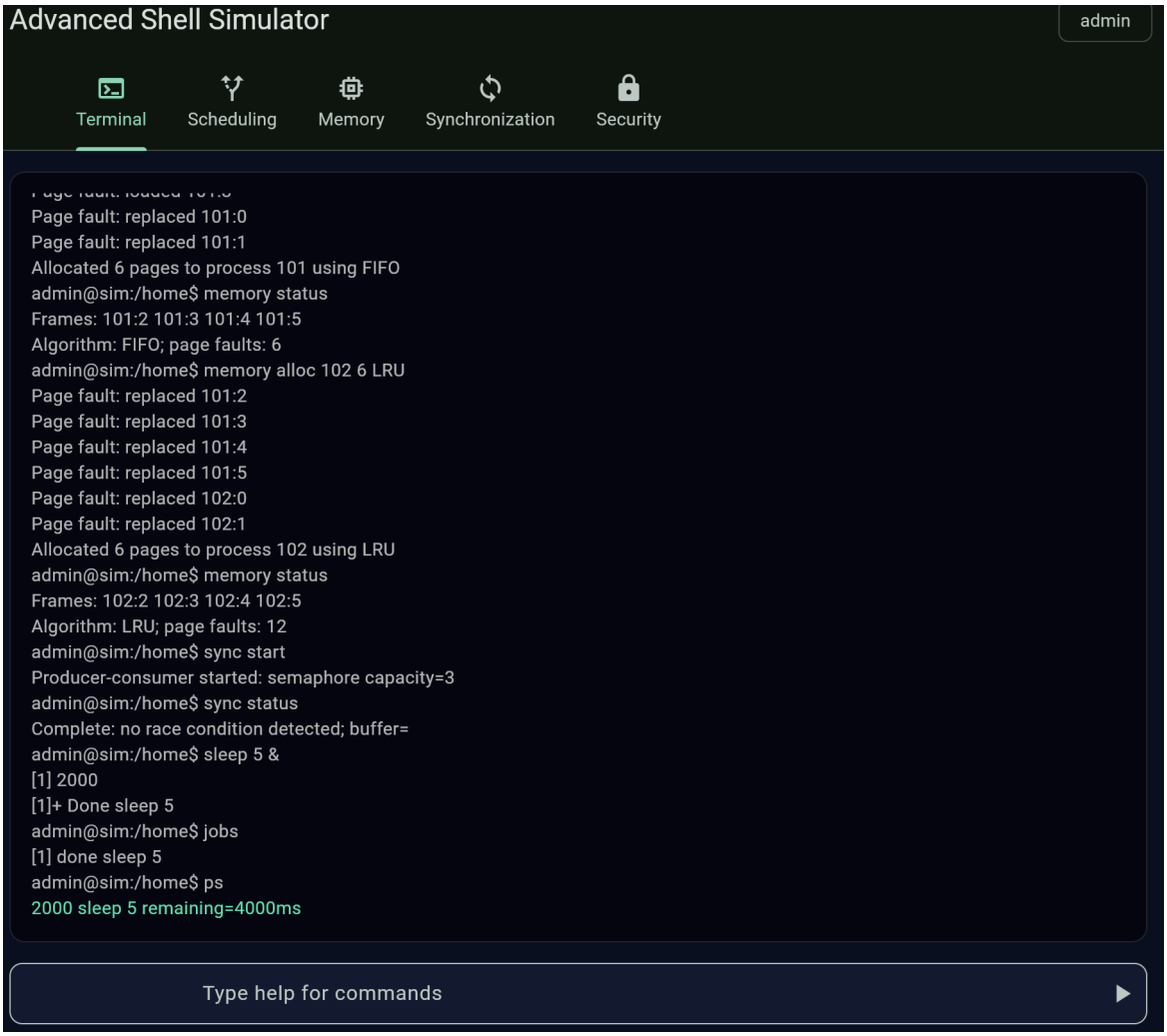


What this screenshot demonstrates

- **Round-Robin:** with a 500 ms quantum, compile, backup, and render take turns until their simulated bursts finish. The output reports execution order, average waiting time, and response times.
- **Priority scheduling:** the sample processes are listed from highest to lowest priority, with calculated waiting and turnaround values. This demo orders the fixed process set; it does not show a time-sliced preemption trace.
- **FIFO paging:** allocating six pages with four frames causes six page faults; the oldest resident pages are evicted as the later pages arrive. The status output shows the final resident pages.
- **LRU paging:** a second allocation uses least-recently-used replacement. The cumulative fault counter and frame listing make evictions and resident-page changes observable.

3. Memory state, synchronization, and background jobs

This screenshot captures the continuation of the memory test and two additional integrated OS-concept demonstrations.



What this screenshot demonstrates

- **Memory:** the final status shows four resident pages for process 102 and a total of 12 page faults after both six-page allocation examples.
- **Synchronization:** `sync start` runs the bounded producer-consumer demonstration with a capacity of three. The later status reports completion and an empty buffer after consumption.
- **Job control:** `sleep 5 &` creates a simulated background job with a synthetic process ID. `jobs` reports the job as done after its timer completes.
- **Simulation note:** jobs and process IDs are modeled in app memory; no real operating-system process is launched. The retained `ps` row is a simulator record, not evidence of a live process.